RICE UNIVERSITY

3D Matrix Multiplication and Tensor Parallelism

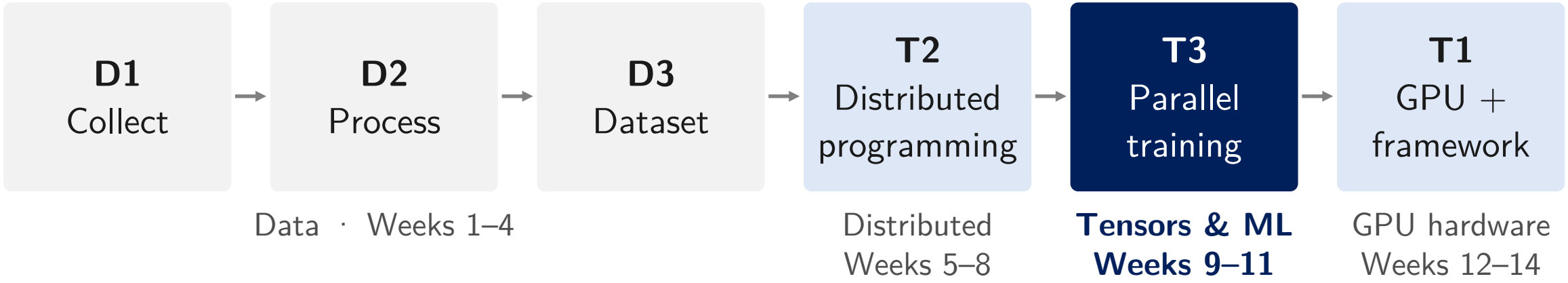
Spending memory to send less, and where tensor parallelism sits on that trade-off

Yuke Wang

Rice University

COMP 329 · Spring 2027 · Week 11, Lecture 2

Where we are in the pipeline



In our pipeline

Today: spend **memory** to send less, and place tensor parallelism on that trade-off. **Lab 5 is due** after this lecture.

Lecture 1: 1D and 2D grids → today: a 3D grid, and tensor parallelism as 1D or 2D

Why do we need a 3D algorithm?

Lecture 1, 2D layout: 64 GPUs, $n = 65{,}536$
arithmetic $\approx$ 9 ms, transfer $\approx$ **43 ms**; memory used $\approx$ **0.4 GB** of 80

Most of each GPU's memory is idle. Can it buy less communication?

2D grid
memory 0.4 GB · transfer $\approx$ 43 ms

3D grid
memory 1.6 GB · transfer $\approx$ 32 ms

Answer

Yes. Storing bigger blocks lets each processor do more work per number it receives.

Source: NVIDIA DGX H100 User Guide (400 Gb/s per GPU); H100 datasheet (SXM5). Leading-order counts.

The 3D algorithm, in plain words

- $\mathbf{p = q^3}$ processors in a q $\times$ q $\times$ q cube; P(i, j, k) computes **one** block product A(i,k) B(k,j).
- Copy each input block to the q processors that need it (**all-gather**).
- Add the q partial products of each C(i,j) (**reduce-scatter**).

Key idea

Lecture 1 looped over k **in time**; the 3D algorithm spreads k over **processors**.

Source: Agarwal, Balle, Gustavson, Joshi & Palkar, "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev. 39(5), 1995.

Where it shows up

- **1995, Agarwal et al. (IBM)**: the 3D algorithm with all-gather and reduce-scatter.

- **2011, 2.5D**: keep c copies, anywhere between 2D and 3D.
- **Deep learning**: 1D (Megatron), 2D, 2.5D and 3D **tensor parallelism**.
- **In this course**: Lab 5 and T3 use 1D; T3 asks you to justify a layout under a memory budget.

In our pipeline

Why 3D? When memory is spare, it can replace communication — and tensor parallelism is one point on that trade-off.

Source: Agarwal, Balle, Gustavson, Joshi & Palkar, "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev. 39(5), 1995.

Learning goals for today

- **Run** the 3D algorithm: all-gather, multiply, reduce-scatter on a $q \times q \times q$ grid.
- **Trade** memory for communication: compare 1D, 2D and 3D per processor.
- **Explain** the lower bound: work is a volume, communication a surface.
- **Read** column/row tensor parallelism as 1D, and 2D tensor parallelism as SUMMA.

A puzzle to start

Lecture 1's 2D grid

p processors, memory $3n^2/p$ each

words received $\approx 2n^2/\sqrt{p}$

Same p, 4× the memory

each processor may store

$4 \times 3n^2/p$ numbers

By how much can the words each processor receives fall: 4×, 2×, or not at all?

Answer

2×. Memory helps under a **square root**: c times the memory $\rightarrow$ $\sqrt{c}$ times fewer words.

Source: Solomonik & Demmel, "Communication-optimal parallel 2.5D matrix multiplication," Euro-Par 2011.

Recap: 1D and 2D from Lecture 1

Per processor	words received	memory
1D strips	$\approx n^2$	$3n^2/p$
2D grid (Cannon, SUMMA)	$\approx 2n^2/\sqrt{p}$	$3n^2/p$

Lecture 1

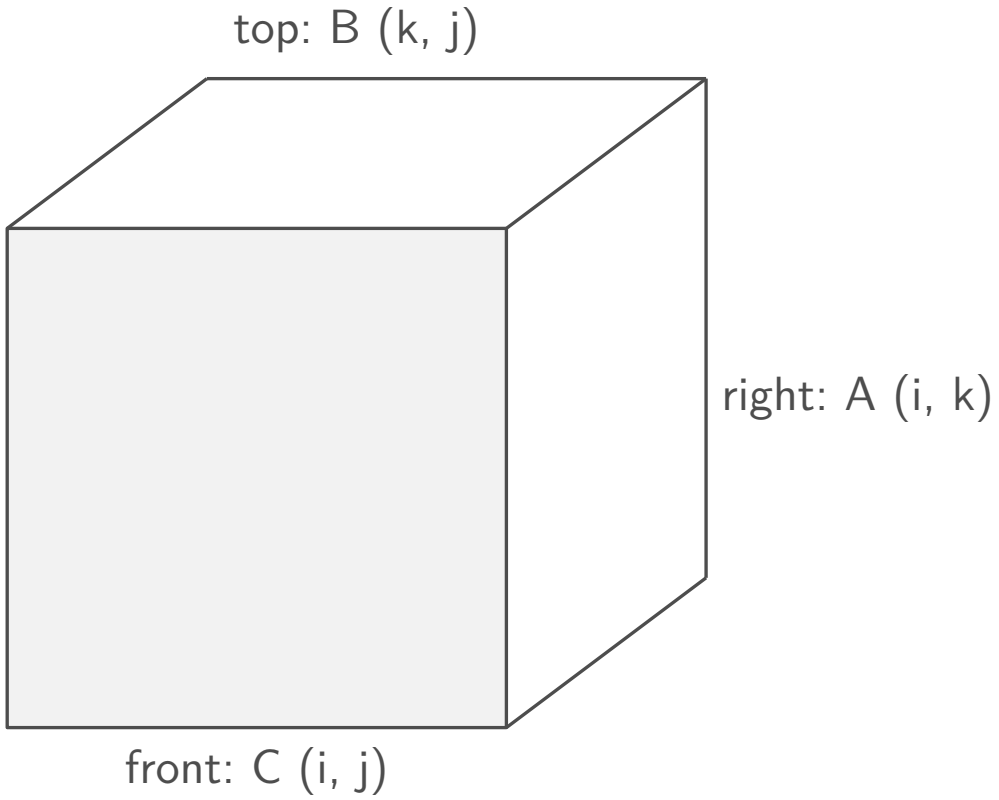
Both keep the **minimum** memory. Today: what if memory is allowed to grow?

Part 1

The 3D algorithm

Give every inner index k its own processors

The cube of multiply-adds

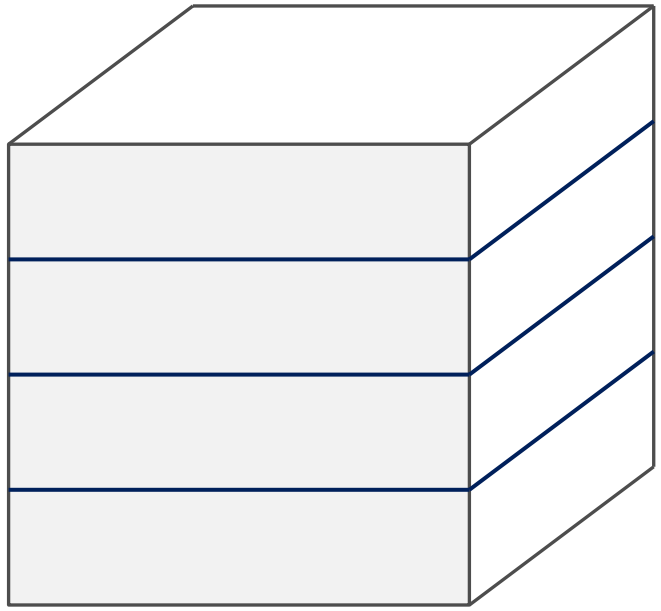


- One point = one multiply-add a(i,k) · b(k,j): $\mathbf{n^3}$ of them.
- Each **face** is a matrix; the C face sums along the depth k.
- A processor's part of the cube needs its **shadows** on the A and B faces.

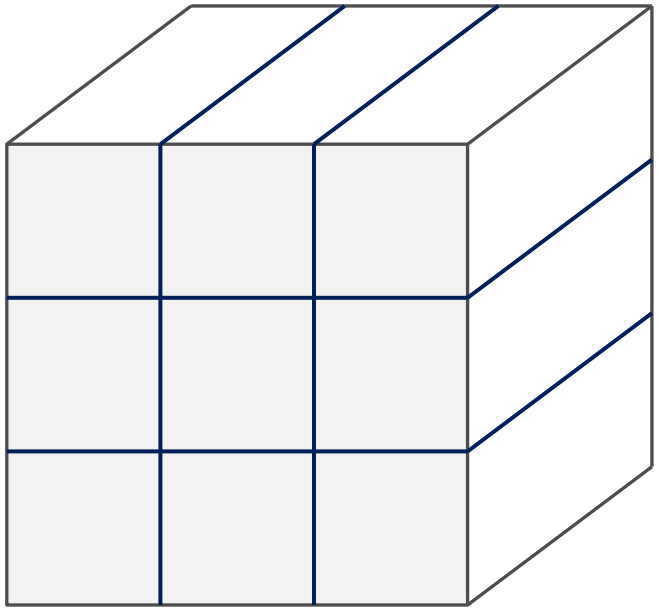
Key idea

An algorithm = a way to **cut the cube**. What a part needs = its shadows on the faces.

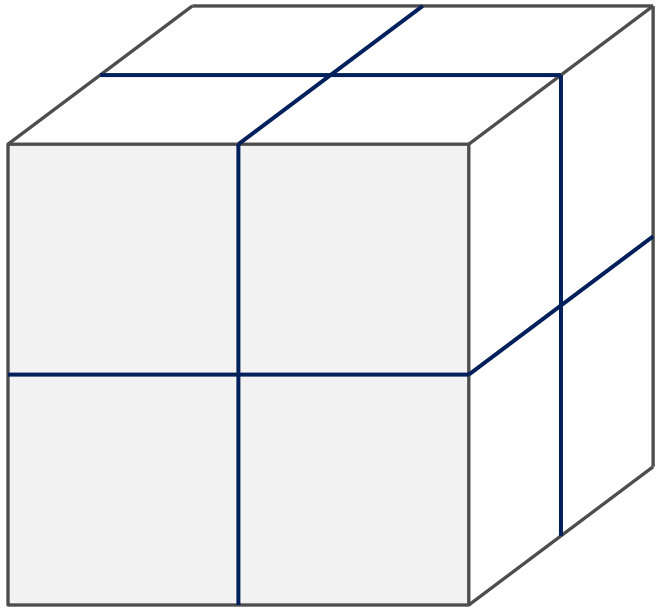
1D, 2D and 3D cut the cube



1D: slabs



2D: rods

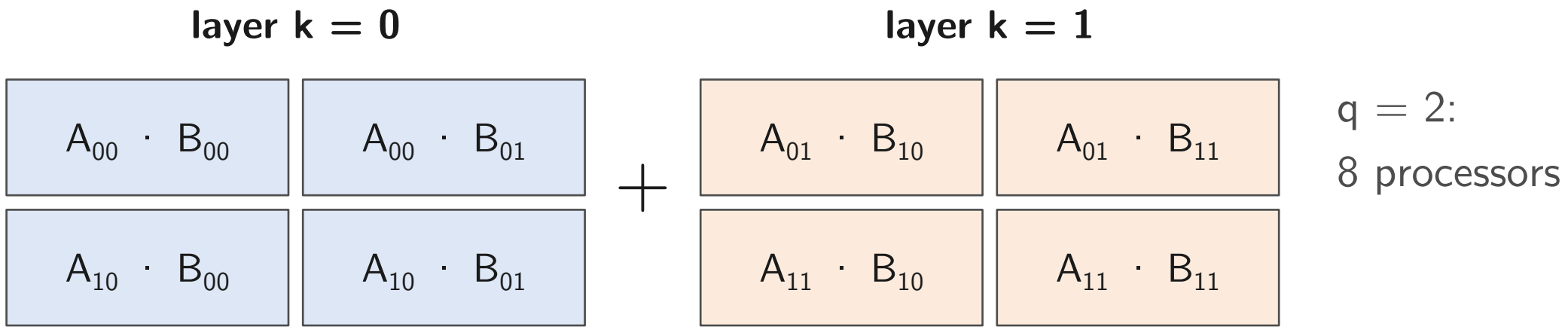


3D: small cubes

Key idea

Same **volume** (work) for every part; the **shape** decides how much it must receive.

The 3D processor grid



q = 2:
8 processors

$$C(i,j) = A(i,0)\ B(0,j) + A(i,1)\ B(1,j)$$

Key idea

P(i, j, k) does **one** block product; the q layers hold the q terms of every C(i,j).

Source: Agarwal, Balle, Gustavson, Joshi & Palkar, "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev. 39(5), 1995.

Three collectives

1. All-gather A(i,k)
over the q processors P(i, $\cdot$, k)

2. All-gather B(k,j)
over the q processors P($\cdot$, j, k)

3. Multiply
A(i,k) B(k,j): $2n^3/p$ FLOPs, no messages

4. Reduce-scatter C(i,j)
sum over k across P(i, j, $\cdot$)

Week 7

All-gather: everyone ends with **all** the pieces.

Reduce-scatter: everyone ends with **one piece of the sum**.

Key idea

Copy the inputs, multiply locally, **sum along k**.

Source: Agarwal, Balle, Gustavson, Joshi & Palkar, "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev. 39(5), 1995.

Counting the 3D algorithm

$$\text{words:}\quad 3 \cdot n^2/q^2 \qquad (p = q^3)$$

$$\text{messages:}\quad \approx 3 \log_2 q \qquad (2D\text{: } 2\sqrt{p})$$

$$\text{memory:}\quad 3n^2/q^2 \;=\; q \times (3n^2/p)$$

Watch out

3D sends fewer words and far fewer messages — and stores every block **q times**.

Source: Agarwal, Balle, Gustavson, Joshi & Palkar, "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev. 39(5), 1995. $p = q^3$.

Check yourself

Two minutes with a neighbour.

- $p = 64 = 4^3$: words and memory per processor in 3D? In 2D?
- Why does the 3D algorithm need a reduction, when Cannon did not?
- How many processors hold a copy of the block A(i,k)?

Part 2

Memory versus communication

How much can memory buy, and where does it stop?

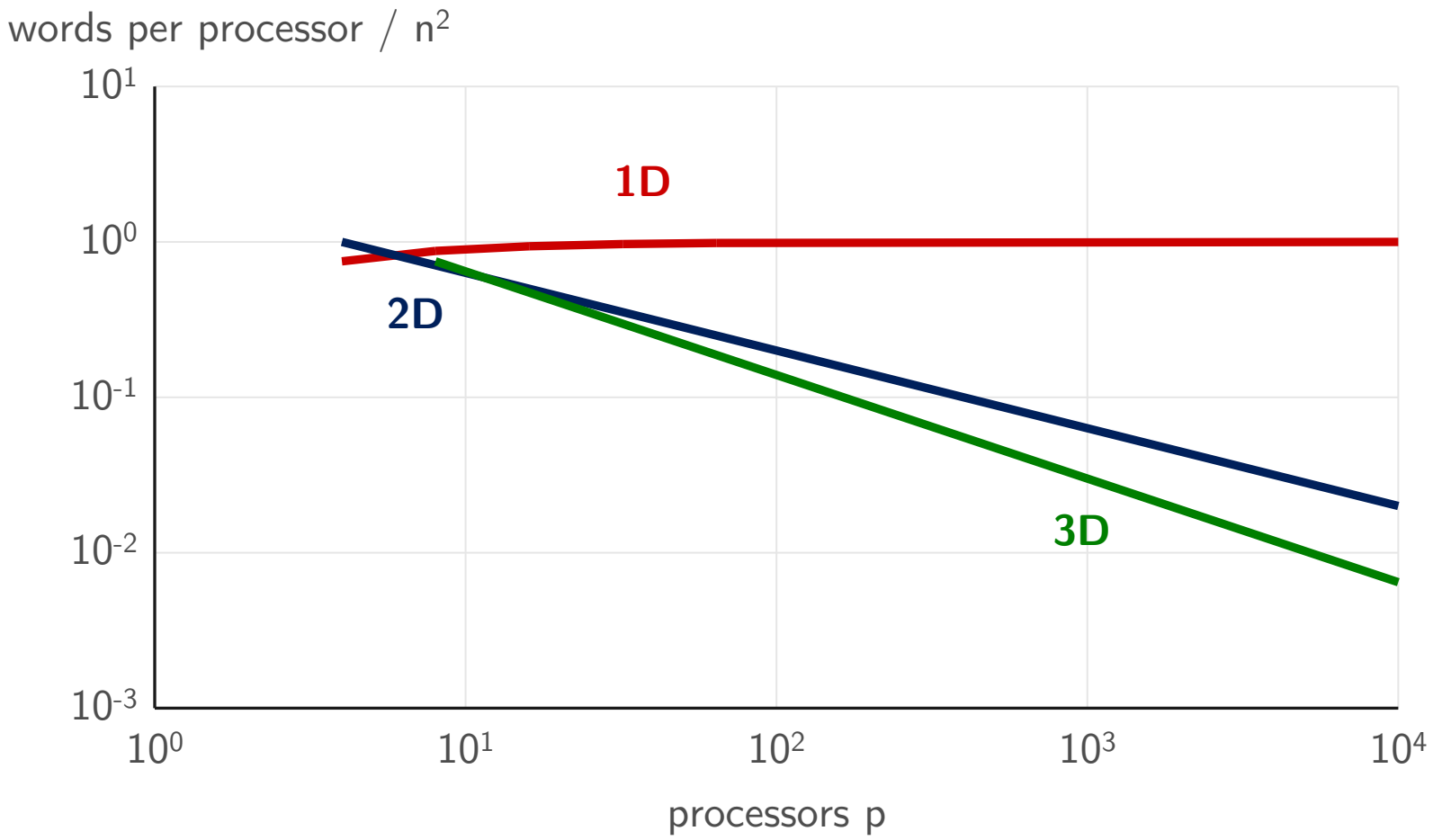
1D, 2D and 3D side by side

Per processor	words received	messages	memory
1D strips	$\approx n^2$	$p - 1$	$3n^2/p$
2D grid	$\approx 2n^2/\sqrt{p}$	$2\sqrt{p}$	$3n^2/p$
3D grid, $p = q^3$	$\approx 3n^2/q^2$	$\approx 3 \log_2 q$	$3n^2/q^2$

Key idea

1D $\rightarrow$ 2D: fewer words **for free**. 2D $\rightarrow$ 3D: fewer still, paid for with **$q\times$ the memory**.

Words per processor as p grows



slopes on log–log axes:

1D: 0

2D: $-1/2$

3D: $-2/3$

...but 3D stores q× more.

Why the cube wins: volume and surface

Part of the cube	shape	shadows (words)
1D slab	$n/p \times n \times n$	$\approx n^2$
2D rod	$n/\sqrt{p} \times n/\sqrt{p} \times n$	$\approx 2n^2/\sqrt{p}$
3D cube	$n/q \times n/q \times n/q$	$3n^2/q^2$

Key idea

Work is a **volume**, communication a **surface**: for a fixed volume, a **cube** has the least surface.

Source: Loomis & Whitney, 1949; Ballard et al., SPAA 2012 (memory-independent bound).

The lower bound with limited memory

$$\text{words} \gtrsim n^3 \,/\, (\, p \cdot \sqrt{M}\,)$$

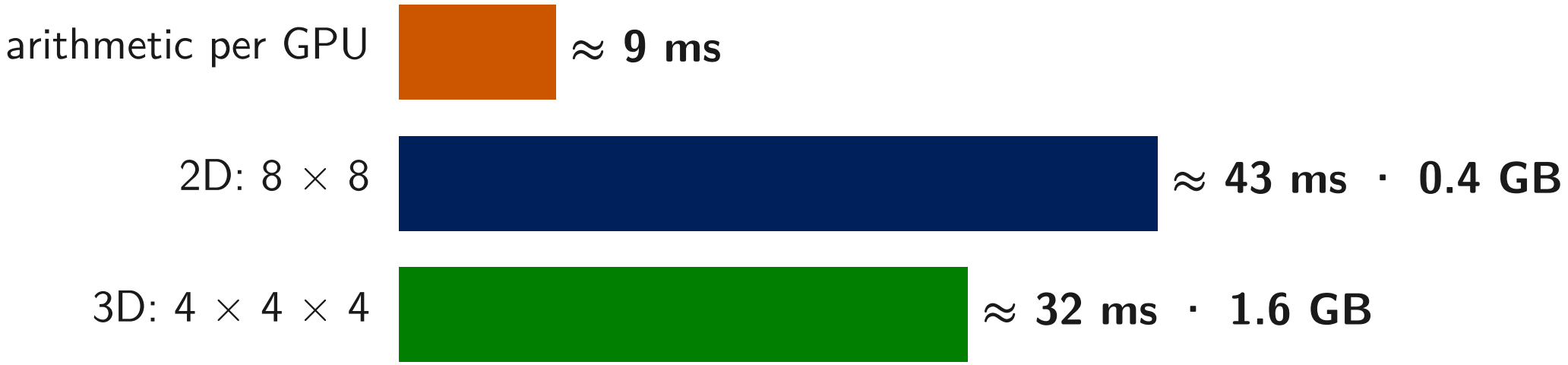
- M = $3n^2/p$ (2D): bound $\propto n^2/\sqrt{p}$ — **Cannon and SUMMA are optimal**.
- M = $3n^2/q^2$ (3D): bound $\propto n^2/q^2$ — **3D is optimal**.
- c copies, M = $3cn^2/p$: bound $\propto n^2/\sqrt{(cp)}$ — the **2.5D** algorithm.

Check

Memory enters under a **square root**: 4× the memory → **2×** fewer words (the puzzle).

Source: Irony, Toledo & Tiskin, "Communication lower bounds for distributed-memory matrix multiplication," JPDC 2004. 2.5D: Solomonik & Demmel 2011.

Back to the opening example



Key idea

3D: ≈ **25% less** transfer for **4×** the memory at p = 64. The gain grows with p; the memory bill does too.

Source: NVIDIA DGX H100 User Guide (400 Gb/s per GPU); H100 datasheet (SXM5). Bandwidth terms only.

Check yourself

Two minutes with a neighbour.

- Why does memory beyond 3D's not reduce communication further?
- A 2D algorithm gets $9\times$ the memory per processor. How many times fewer words, at best?
- Which part of the cube should a processor own to read the fewest words?

Part 3

Tensor parallelism as a decomposition

Week 10's splits are cuts of the cube

Recap: Week 10's two splits

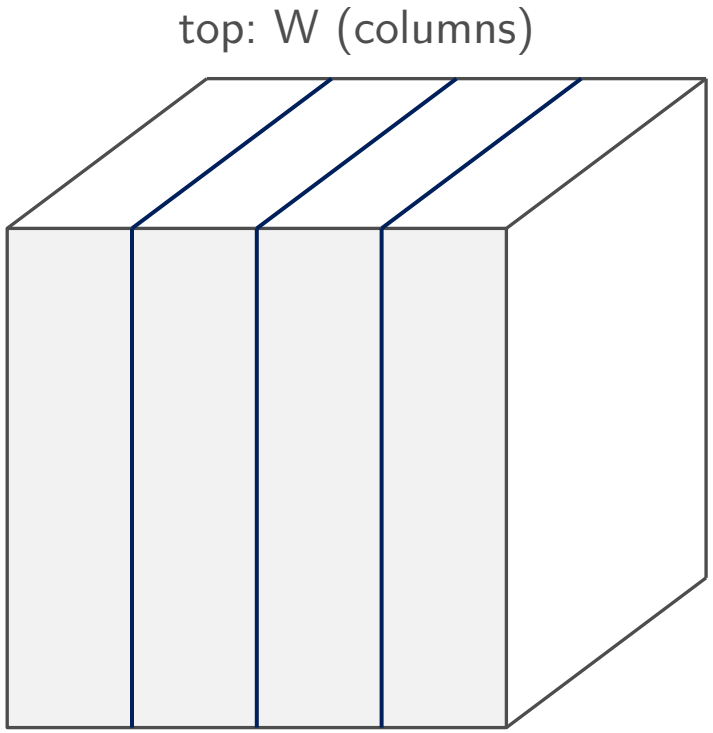
- A layer: $\mathbf{Y = X\ W}$; X activations (m $\times$ k), W weights (k $\times$ n).
- **Column split** of W: slices of Y; X **whole** on every GPU.
- **Row split** of W: partial sums of Y $\rightarrow$ one **all-reduce**.

Week 10

Megatron: column, then row — **one all-reduce** per block, inside one server.

Source: Shoeybi et al., "Megatron-LM," 2019.

The column split is a 1D cut

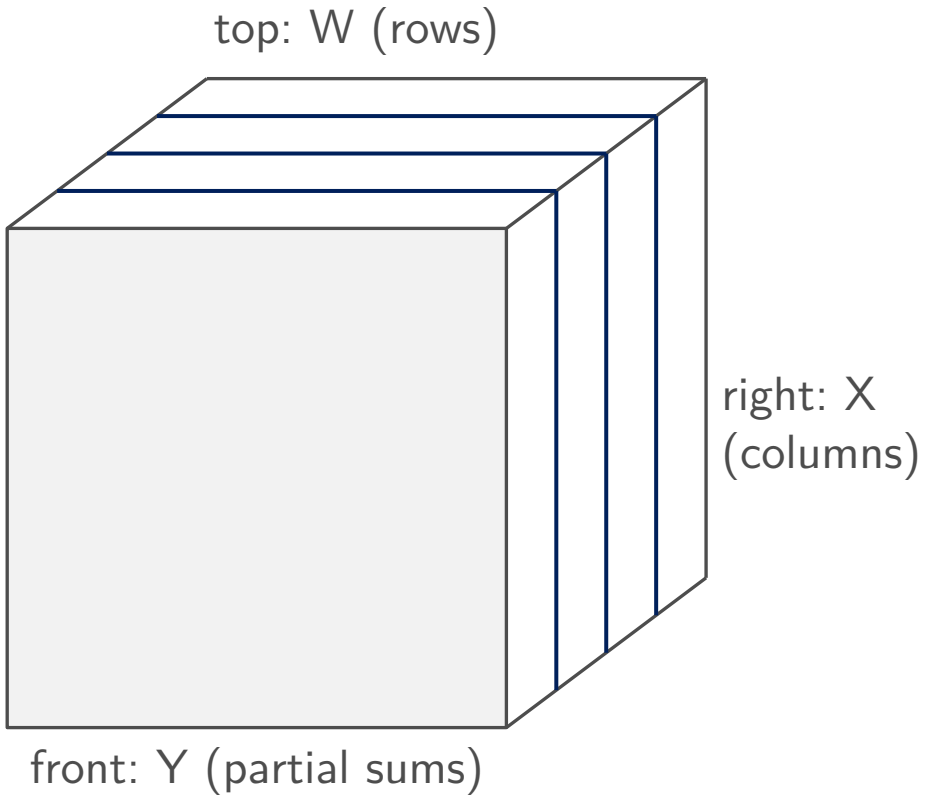


- Cut along **j** (output columns) only: a **1D** cut.
- Shadow on X: the **whole** face → X on every GPU.
- Shadow on W: a slice → weights **split** N ways.

Key idea

Column split = Lecture 1's **1D strips**: every GPU needs **all** of the activations X.

The row split is a 1D cut along k



- Cut along **k** (the inner dimension): also **1D**.
- Every GPU's slab covers **all of Y**: a partial sum.
- Cutting k always needs a **sum** — like 3D's reduce-scatter.

Key idea

A cut across **k** means a reduction; a cut across i or j means a **copy** of an input.

What 1D tensor parallelism costs

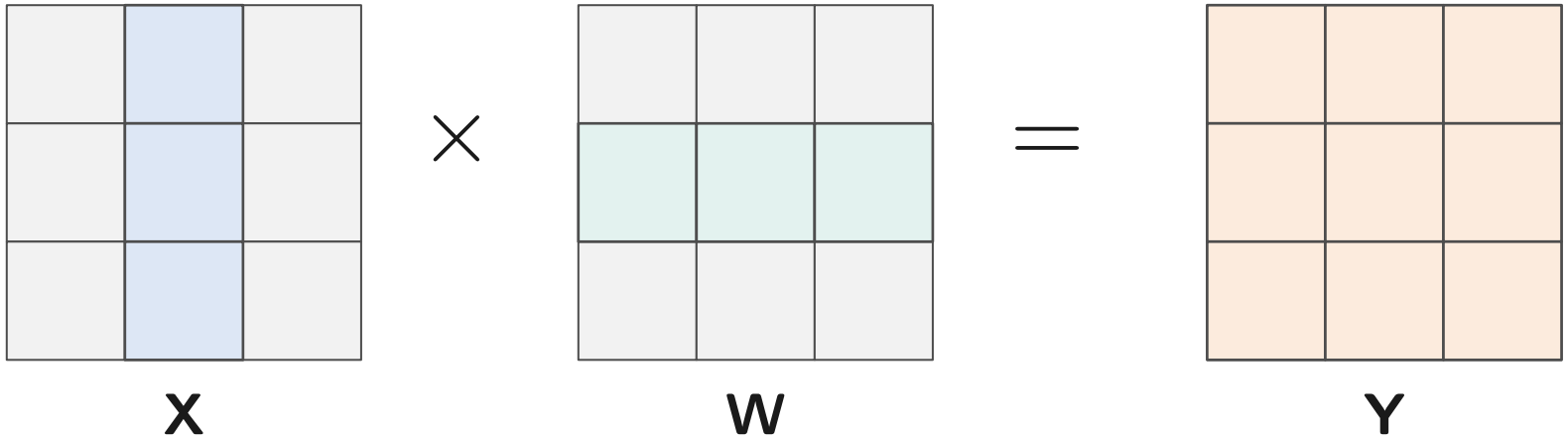
Per GPU, 1D (N GPUs)	shrinks with N?
weights: kn / N	yes
activations X, Y: $mk + mn$	no
all-reduce: $\approx 2mn$ words	no

Watch out

Like Lecture 1's strips: per-GPU communication **does not shrink** with N. Fine for $N \leq 8$ inside a server.

Source: Shoeybi et al., "Megatron-LM," 2019. Week 10, Lecture 1.

2D tensor parallelism is SUMMA



every block of Y:
+= X(i,k) W(k,j)

X(i,k) along rows

W(k,j) along columns

Key idea

N = q^2 GPUs, X and W both in blocks: Y = X W **by SUMMA**. No replicated activations; **weights move**.

Source: Xu et al., "An Efficient 2D Method for Training Super-Large Deep Learning Models" (Optimus), 2021.

1D versus 2D tensor parallelism

Per GPU	1D (column/row)	2D (SUMMA)
weights stored	kn / N	kn / N
activations stored	$mk + mn$	$(mk + mn) / N$
words sent	$\approx 2mn$	$\approx (mk + kn) / \sqrt{N}$

Key idea

2D shrinks **activations** and communication with N, but sends **weights**: it pays at **large N**.

Source: Xu et al., "An Efficient 2D Method for Training Super-Large Deep Learning Models" (Optimus), 2021.

Tensor parallelism on the trade-off

- **Inside a server** ($N \leq 8$): 1D — small N, fast links, weights stay put.
- **Large N** or huge activations: 2D, 2.5D, 3D tensor parallelism.
- Across servers: **pipeline** and **data** parallelism (Week 10, Lecture 2).

In our pipeline

Tensor parallelism is **one point** on the memory–communication trade-off; 1D wins at small N.

Source: Shoeybi et al., "Megatron-LM," 2019. Narayanan et al., SC 2021.

Check yourself

Two minutes with a neighbour.

- Which axis of the cube does the column split cut? The row split? What does each cost?
- Why does 1D tensor parallelism's communication per GPU not shrink as N grows?
- What does 2D tensor parallelism send that 1D never does?

IN-CLASS BUILD

Choose a layout

Work in pairs: given p and a memory budget, pick 1D, 2D or 3D.

Exercise: 1D, 2D or 3D?

Case	p	memory budget	words: 1D / 2D / 3D	choice
A	64	0.1 S		
B	4,096	0.1 S		
C	4,096	0.005 S		

USE

Words: n^2, $2n^2/\sqrt{p}$, $3n^2/q^2$. Memory: $3n^2/p$, $3n^2/p$, $3n^2/q^2$. $p = q^3$. $S = n^2$.

STRETCH

Case C: how many copies c would a 2.5D layout allow, and how many fewer words than 2D?

Exercise: solutions

Case	p	memory budget	words: 1D / 2D / 3D	choice
A	64	0.1 S	0.98 / 0.250 / 0.188*	**2D**
B	4,096	0.1 S	1.00 / 0.031 / 0.012	**3D**
C	4,096	0.005 S	1.00 / 0.031 / 0.012*	**2D**

words in units of $S = n^2$; * = memory over budget

Check

Stretch (C): 2.5D with $\mathbf{c = 4}$ copies fits, and sends $\mathbf{2\times}$ fewer words than 2D.

How to choose, in general

- **Small p** (one server): the layouts differ little $\rightarrow$ **1D**, the simplest.
- **Larger p**: a **2D** grid — $\sqrt{p}/2$ times fewer words, same memory.
- **Spare memory**: replicate — c copies buy $\sqrt{\mathbf{c}}$ fewer words, up to **3D**.

Key idea

Spend memory you **do not need** on communication you **can avoid** — with diminishing returns.

Key takeaways

- 3D on $p = q^3$: **all-gather** A and B, multiply, **reduce-scatter** C; $\approx \mathbf{3n^2/q^2}$ words.
- It stores every block **q times** ($p = q^3$): memory buys communication.
- Lower bound $\approx \mathbf{n^3/(p\sqrt{M})}$: memory helps under a square root; 2D and 3D are optimal for theirs.
- Column and row splits are **1D cuts**; per-GPU traffic does not shrink with N.
- **2D tensor parallelism = SUMMA**: activations shrink, weights travel; it pays at large N.

Next time, and readings

When	What
After this lecture	Lab 5 due; T3 checkpoint this week
Week 12, Lecture 1	GPU architecture
Week 12	T3 due Friday; T1 and Lab 3 out after Lecture 2

READING

Agarwal et al., "A three-dimensional approach to parallel matrix multiplication," IBM J. Res. Dev., 1995 (§1–3).

For Week 12: CUDA C++ Programming Guide, *Introduction* and *Programming Model*.